III.3 Langages de communication et d'échange d'information sémantisée entre agents

Dans cet exposé, nous cherchons à donner un aperçu des langages de communication entre agents. Nous commencerons par introduire **les ACLs et les actes de langage**, avant de voir les **débuts** de ces fameux ACLs. Mais communiquer dans un langage commun ne suffit pas : il faut aussi partager une ontologie, c'est à dire une représentation du monde. Nous verrons alors **quels langages le W3C propose** pour décrire des ontologies. Nous conclurons enfin par un rapide récapitulatif des **équipes** travaillant sur le sujet et une petite **bibliographie**.

Introduction : ACLs et actes de langages

Les **agents** sont des composants logiciels caractérisés par leur **autonomie**, leur **adaptativité** et leur **caractère coopératif**.

Depuis quelques années, on a cherché à permettre aux programmes informatiques de s'échanger des informations et des connaissances [**IEEE99**].

Les techniques suivantes ont déjà été utilisées, indépendamment des notions d'"agent" :

- **RPC** (Remote Procedure Call) : il s'agit d'appeler des procédures à distance à travers un réseau. Ce protocole date de la grande époque d'UNIX, où il s'agissait de faire communiquer des programmes impératifs typiquement écrits en C. Ces mécanismes sont bien au point, mais s'intègrent mal dans les modèles de programmation par objets en vogue actuellement ;
- **RMI** (Remote Method Invocation) : c'est un mécanisme d'appel de méthode sur des objets **Java**, entre programmes Java, qui fonctionne grâce au réseau. Bien qu'intéressant, ce mécanisme est cantonné au langage Java ;
- **CORBA** (Common Object Resource Broker Architecture) : proche par son principe de RMI, ce mécanisme est par contre disponible sur de nombreux langages de programmation. Il permet l'invocation de méthodes à distance et l'échange d'objets sur le réseau.

Cependant, ces techniques sont limitées car elles restent cantonnées au cadre de la programmation impérative ou procédurale : à chaque instant, le programme sait quelle procédure appeler. En ce sens, elles sont mal adaptées aux systèmes multi-agents.

C'est pourquoi plus récemment, sont apparus les **langages de communication** entre agents (en anglais : **ACLs**, Agent Communication Languages), qui vont plus loin :

1. ils manipulent des **propositions**, des **règles** et des **actions** au lieu de simples objets sans sémantique associée ;
2. un message décrit un **état** dans un **langage déclaratif**, au lieu d'une simple invocation de méthode ou d'un simple appel de procédure.

Les types de messages des ACLs sont appelés des **actes de langage**, décrits dans la théorie de John Searle [**SEARLE69**]. Il existe plusieurs catégories de tels actes :

- **assertifs** : pour donner une information ;
- **directifs** : pour donner des ordres ;
- **promissifs** : pour s'engager sur une action à venir ;
- **expressifs** : pour exprimer ses sentiments ou ses croyances ;
- **déclaratifs** : pour énoncer un fait ;
- **interrogatifs** : pour demander une information.

En pratique, un message est identifié grâce à son **performatif**, qui correspond à l'une des catégories de Searle.

Par exemple, si le performatif d'un message est `Affirmer`, ce message sera un acte de langage *assertif*. De même, le performatif `OffrirService` correspondra à un acte *promissif*, etc.

Notons cependant que les ACLs ne couvrent pas l'ensemble des messages que des applications pourraient être amenées à s'échanger, notamment des objets plus complexes comme des plans ou des stratégies.

KIF, KQML, ACL FIPA : les débuts

Les premiers travaux de définition d'un ACL standard remontent à 1990, quand la **DARPA** (Defense Advanced Research Project Agency) a initié le **Knowledge Sharing Effort** (**KSE**). En effet, si l'on veut partager la connaissance, il faut s'accorder sur un langage commun : c'est pourquoi le KSE s'est tout de suite tourné vers la conception d'un ACL.

La compréhension entre agents passe par différents niveaux :

1. Tout d'abord, il faut que les agents parlent le même langage, ou bien qu'il existe des traducteurs bijectifs d'un langage dans un autre. On est ici au **niveau syntaxique** ;
2. Mais il faut également que les mêmes objets, les mêmes concepts aient la même signification pour tous les agents. Il est donc nécessaire que tous les agens partagent une **ontologie** commune, c'est à dire une définition des concepts associée aux relations qui existent entre eux ;
3. Enfin, les agents doivent être capables de dialoguer entre eux, pas au niveau du simple envoi de messages, mais à un niveau supérieur : ils doivent pouvoir s'informer, se poser des questions, rechercher d'autres agents, etc. Pour ce faire, les agents ne peuvent pas se contenter d'appeler une procédure (laquelle appelleraient-ils...?) ; ils doivent spécifier un *état désiré* dans un **langage déclaratif** : c'est ici qu'intervient l'ACL.

KIF : Knowledge Interchange Format

Le niveau purement syntaxique (niveau 1) peut être pris en charge par **KIF** (Knowledge Interchange Format), dont la syntaxe est inspirée de **Lisp**. En effet, un message KIF peut être vu comme une **s-expression**, une s-expression étant elle-même soit un atome, soit une liste de s-expressions entourée de parenthèses.

KQML : Knowledge Query and Manipulation Language

Le langage **KQML** se positionne au 3ème niveau de notre classification. Il est complètement indépendant à la fois de la syntaxe et de l'ontologie sous-jacente.

Un message KQML se compose :

- d'un **performatif**, qui représente l'*en-tête* du message ;
- d'un certain nombres de **couples attribut-valeur**.

Voici quelques exemples de performatifs, parmi les 37 définis dans la norme de 1997 :

Performatif	Catégorie de Searle	Signification
Tell	assertif	L'agent émetteur indique qu'il considère que le contenu du message est vrai
Ask-if	interrogatif	L'agent émetteur demande à l'agent récepteur s'il considère que le contenu du message est vrai
Advertise	promissif	L'agent émetteur promet qu'il exécutera le code contenu dans le message si jamais il le reçoit à l'évenir

Voyons maintenant quels sont les attributs les plus courants :

Attribut	Signification
:content	Contenu du message
:language	Langage dans lequel le contenu est exprimé
:ontology	Ontologie sous-jacente nécessaire pour comprendre le contenu du message
:sender	Nom de l'émetteur du message
:receiver	Nom du récepteur du message

D'un point de vue plus théorique, on peut considérer qu'un message KQML se compose de trois couches :

- couche de **contenu** : le contenu du message, c'est à dire la valeur de l'attribut `:content` ;
- couche de **communication** : informations qui permettent l'acheminement du message, par exemple les valeurs des attributs `:sender` et `:receiver` ;
- couche de **message** : composée du performatif et des valeurs des attributs qui décrivent le message, comme `:language` ou `:ontology`.

Voyons maintenant un exemple de message KQML dont le contenu est exprimé en KIF. Notons que le message KQML est lui-même écrit dans une syntaxe de type Lisp :

```
(ask-one
  :sender bob
  :receiver bill
  :language KIF
  :ontology DEA-I3
  :reply-with q1
  :content (val (mark bob))
)
```

L'agent `bob` demande (`ask-one`) à l'agent `bill` de lui donner sa note. Le langage de contenu est KIF, avec l'ontologie `DEA-I3`. L'attribut `reply-with` permettra à `bob` d'identifier la réponse à ce message précis lorsqu'il verra un message étiqueté par `:in-reply-to q1`, par exemple :

```
(tell
  :sender bill
  :receiver bob
  :langage KIF
  :ontology DEA-I3
  :in-reply-to q1
  :content (= (mark bob) 18)
)
```

La FIPA et son ACL

La **FIPA** (Foundation for Intelligent Physical Agents) est une association à but non lucratif qui a pour ambition le développements des systèmes à agents. L'adjectif *physical* est là pour rappeler que les agents peuvent éventuellement être des humains. Les grandes sociétés de télécoms sont membres de la FIPA.

En 1999, la FIPA a défini un ACL [**FIPA00061**] pour répondre principalement à une critique assez vigoureuse contre KQML : ses performatifs sont trop nombreux, donc redondants, et de plus ils ne couvrent pas en entier le champ des performatifs envisageables.

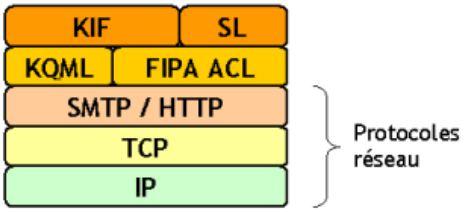
L'ACL de la FIPA ressemble beaucoup à KQML, mais il ne comporte que 22 performatifs. De plus, il est défini de façon beaucoup moins ambigüe : la sémantique des performatifs est décrite [**FIPA00037**] dans un langage spécialisé, le langage **SL** pour **Semantic Language** (d'après les travaux de Sadek).

SL est une logique multi-modale qui comporte différents quantificateurs :

- **B** : “je crois que” (*belief*) ;
- **D** : “je désire” (*desire*) ;
- **U** ; “je suis incertain au sujet de” (*uncertainty*).

SL est donc utilisé pour décrire la sémantique formelle de l'ACL (avec une syntaxe mathématique). Mais il peut revêtir une syntaxe de type Lisp, de façon a être utilisé comme langage de contenu des messages eux-mêmes [**FIPA00008**] à la place de KIF.

Conclusion sur les ACLs



Le schéma ci-contre montre la superposition des protocoles et langages de communication entre agents. Les langages de communication entre agents (comme KQML ou l'ACL de la FIPA) s'appuient sur les protocoles de communication classiques d'Internet : **SMTP** (Simple Mail Transfer Protocol) ou **HTTP** (Hypertext Transfer Protocol), qui reposent eux-mêmes sur les couches **TCP** (Transmission Control Protocol) et **IP** (Internet Protocol).

Jusqu'en 1999, les langages de communication entre agents avaient été développés de façon complètement indépendante d'Internet. Notamment, aucun lien n'avait été fait avec les technologies du Web sémantique comme XML et/ou RDF. Les choses ont beaucoup changé depuis, notamment avec les travaux du W3C.

L'entrée en scène du W3C : ontologies

Le Web et HTML

Le Web a été conçu en 1990 au **CERN** par **Tim Berners-Lee** et Robert Cailliau. Il s'agissait de permettre la navigation par hypertexte dans un ensemble de documents. À ces fins, l'équipe a développé un protocole de communication (**HTTP**, Hypertext Transfer Protocol) et un dialecte **SGML** (Standard Generalized Markup Language) pour décrire les documents (**HTML**, Hypertext Markup Language). D'abord utilisé en interne au CERN, le système s'est rapidement répandu sur Internet.

Le **W3C** est le World Wide Web Consortium. Cette organisation à but non lucratif a été fondée en octobre 1994, suite à la croissance exponentielle du Web conséquente à la diffusion du navigateur **NCSA Mosaic**. Dans les années qui ont suivi, la guerre commerciale entre les navigateurs **Microsoft Internet Explorer** et **Netscape Navigator** (lancé par un ancien du projet Mosaic) a menacé de faire imploser le Web : en effet, le langage HTML, développé de façon propriétaire par chacun des deux concurrents, menaçait de se scinder en deux langages.

C'est pourquoi deux des sept buts principaux du W3C [**W3C-7**] sont l'accès universel aux informations et l'interopérabilité. Le W3C a donc réagi à la guerre des navigateurs en définissant un HTML standard, que tout logiciel se disant “navigateur Web” devrait respecter.

XML

Mais on s'est rapidement rendu compte que le HTML, même normalisé, a tendance à privilégier la forme des documents sur leur fond. En effet, il s'agit d'un langage à balises, et ces balises décrivent la structure d'un document (niveaux de titres, paragraphes, etc.).

Pour décrire le sens profond des documents, il faudrait pouvoir se définir un vocabulaire de description propre à chaque type de documents : en effet, on n'utilise pas le même méta-langage pour décrire une recette de cuisine ou une **formule mathématique** !

Avec SGML, qui existait depuis le début des années 1980, il était possible de définir le vocabulaire de description des documents au sein de DTDs (Document Type Definitions). Cependant, SGML était trop complexe. C'est pourquoi le W3C a produit une version simplifiée de SGML : **XML** (eXtensible Markup Language). Ce langage a fêté ses **cinq ans d'existence** le 10 février 2003.

En XML, les balises sont redéfinissables. Il est donc possible d'adapter le vocabulaire de description au domaine précis concerné par le document. La définition des balises et leurs relations sont données dans une DTD, héritée de SGML.

Métadonnées et Dublin Core

Dès la fin des années 1990, s'est fait sentir le besoin de donner des informations à propos des données du Web, c'est à dire d'associer des **métadonnées** aux pages Web. C'est pourquoi une initiative indépendante du W3C s'est donné pour but de définir un vocabulaire de métadonnées pour la description des pages Web.

Ce projet a donné naissance au **Dublin Core**. Dans ce vocabulaire, on peut donner facilement le titre, l'auteur, ou encore le sujet d'un document. Il est également possible de définir des relations entre documents.

RDF

Parallèlement au Dublin Core, le W3C a lui aussi cherché à définir un format de méta-données. Cependant, au lieu de définir un vocabulaire précis adapté à un usage précis comme l'avait fait le Dublin Core, le W3C a défini un cadre général pour la définition de toutes sortes de méta-données. Il s'agit du **RDF** (Resource Description Framework), langage de description d'informations sur le Web [**W3-RDF-PRIMER**].

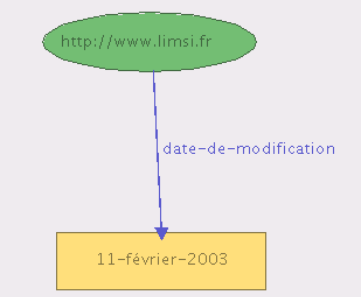
Le modèle RDF définit trois types d'objets [**W3-RDF-CONCEPTS**] :

- des **ressources** : les ressources sont tous les objets *décrits* par RDF. Généralement, ces ressources peuvent être aussi bien des pages Web que tout objet ou personne du monde réel. Les ressources sont alors identifiées par leur **URI** (Uniform Resource Identifier) ;
- des **propriétés** : une propriété est un attribut, un aspect, une caractéristique qui s'applique à une ressource. Il peut également s'agir d'une mise en relation avec une autre ressource ;
- des **valeurs** : les valeurs en question sont les valeurs particulières que prennent les propriétés.

Ces trois types d'objets peuvent être mis en relation par des **assertions**, c'est à dire des triplets (*ressource*, *propriété*, *valeur*), ou encore (*sujet*, *prédicat*, *objet*). Une description RDF est une suite d'assertions.

Voyons trois objets RDF :

- une ressource : `http://www.limsi.fr/` ;
- une propriété : `date-de-modification` ;
- une valeur : `11-février-2003`.



On peut alors construire l'assertion : `(http://www.limsi.fr/, date-de-modification, 11-février-2003)`, qui signifie *"la date de modification de la page Web du LIMSI est le 11 février 2003"*.

Il est possible de représenter les descriptions RDF par des graphes. Par exemple, on voit ci-contre le graphe correspondant à l'essertion précédente.

Dans un graphe RDF, on représente par des ellipses les ressources nommées (i.e. les objets qui possèdent des URI), et par des rectangles les littéraux (i.e. les constantes, qui ne possèdent pas d'URI).

Le W3C développe en **Java** le logiciel **IsaViz**, qui permet de concevoir graphiquement des descriptions RDF, et de les exporter sous divers formats, y compris graphiques. Le graphe ci-contre a été réalisé à l'aide d'IsaViz.
À la base, RDF ne fait que définir ce modèle de représentation. La syntaxe n'est pas spécifiée : on pourrait par exemple décider de représenter toutes les descriptions sous forme de triplets entourés de parenthèses. Cependant, RDF étant issu du W3C, il est assez naturel de vouloir le représenter dans des documents XML [**W3-RDF-XML**]. Ainsi, l'**espace de noms** XML `rdf` a été réservé pour l'inclusion de descriptions RDF dans des documents XML.

Voyons comment on peut représenter l'exemple précédent en XML :

```
<rdf:RDF>
  <rdf:Description about="http://www.limsi.fr/">
    <s:date-de-modification>2003-02-11</s:date-de-modification>
  </rdf:Description>
</rdf:RDF>
```

On a vu qu'il est possible de représenter graphiquement des assertions RDF. En fait, une description RDF sous-tend un graphe :

- les éléments noeuds de l'arbre XML (`rdf:Description`), c'est à dire les ressources, correspondent aux noeuds du graphe ;
- les éléments propriétés correspondent aux arcs entre les noeuds.

Dans l'exemple précédent, on utilise deux espaces de noms : `rdf` et `s`. On l'a vu, l'espace de nom `rdf` correspond aux balises utilisées pour décrire les descriptions RDF elles-mêmes. Quant à l'espace de noms `s`, il fait référence au **schéma RDF** utilisé.

En effet, jusqu'à présent, on n'a dit nulle part quels étaient les attributs autorisés, à quelles ressources ils s'appliquaient, quelles étaient leurs valeurs admises... C'est le schéma RDF qui précise tous ces points. Le schéma donne véritablement sa sémantique à la description RDF. Il permet de décrire ressources et propriétés dans un modèle objet : il existe des classes de ressources et des classes de propriétés. Il est possible de définir des relations entre ces différentes entités : relations de subsomption entre classes bien entendu, mais aussi définition de classes par opérations ensemblistes (union, intersection, différence symétrique, etc.). On peut aussi définir des sous-classes par restriction des valeurs des propriétés des super-classes.

En un mot, le schéma définit le vocabulaire utilisé dans une description RDF. On peut imaginer à loisir de nombreux schémas différents, adaptés chacun) un domaine ou à une application spécifique. Il faut noter que les schémas sont eux-mêmes écrits en RDF, en utilisant des balises de l'espace de nom du langage **RDF Schema** [**W3-RDFS**], souvent désigné par le préfixe `rdfs`.

En particulier, le Dublin Core peut être traité comme l'un de ces schémas RDF, un schéma adapté à la description de pages Web.

Un autre schéma RDF en vogue actuellement est **RSS** (RDF Site Summary), qui permet de donner des résumés de sites Web. En particulier, ce format est bien adapté aux sites qui présentent une liste de nouvelles : avec RSS, on peut présenter objectivement cette liste de nouvelles, de façon indépendante d'une quelconque présentation. Par exemple, la **page d'accueil du W3C** peut être obtenue sous forme de **résumé RSS**. Il est également possible de fusionner de multiples sources de données RSS en une seule présentation, ou encore de présenter sur un site Web une source d'information provenant d'un autre site.

DAML+OIL

Les langages de communication entre agents vus précédemment – KQML et l'ACL de la FIPA – permettent aux agents de communiquer dans des langages standardisés (ils sont *interopérables*), mais rien ne garantit qu'ils *comprennent* les données qu'ils s'envoient !

C'est ici qu'intervient le langage **DAML** (DARPA Agent Markup Language) qui permet aux agents de partager de la sémantique. DAML est associé à **OIL** (Ontology Inference Layer), qui est un autre langage de description d'ontologies. Le couple DAML+OIL repose sur RDF [**W3-DAML+OIL**]. Mais avant d'étudier plus en détail ce DAML+OIL, voyons d'abord les caractéristiques d'OIL, puis de DAML.

OIL est un langage de description et d'inférence sur les ontologies, basé sur RDF. Il prend appui sur les **logiques de description**. Il est composé de plusieurs couches :

- le **coeur** : il correspond presque exactement avec le langage RDF Schema [**W3-RDFS**] qui permet de décrire les vocabulaires RDF ;
- **Standard OIL** : il permet de définir la sémantique de façon plus précise, et donne ainsi la possibilité d'utiliser les mécanismes d'inférence (malgré son nom, il ne s'agit *pas* d'une **compagnie pétrolière américaine** !) ;
- **Instance OIL** : il introduit les fonctionnalités des bases de données ;
- **Heavy OIL** : extension à venir.

Quant à lui, le développement de DAML a commencé en août 2000, avec pour but de favoriser le développement du Web sémantique. En octobre de la même année, la spécification [**DAML-ONT**] a été publiée. Tout comme OIL ou RDF Schema, il s'agit d'un langage de description d'ontologies. En ce sens, on peut définir des classes et des propriétés, et les mettre en relation (subsomption, disjonction, etc.).

Ainsi, DAML+OIL cherche à combiner toutes les caractéristiques de DAML, d'OIL, et de RDF Schema. Bien entendu, il est également construit sur RDF. Il permet de modéliser les aspects suivants :

- définition de classes de propriétés ;
- définition de classes de ressources ;
- relations logiques entre classes (disjonction, union, équivalence, etc.) ;
- relations d'héritage entre classes ;
- restriction de propriétés (cardinalité, etc.) et typage ;
- prise en charge des collections (listes) ;
- instanciation de classes de propriétés et de ressouces.

Voici un exemple d'extrait d'ontologie DAML+OIL :

```
<daml:Class rdf:ID="Homme">
  <rdfs:subClassOf> rdf:resource="#Humain">
</daml:Class>

<daml:Class rdf:ID="Femme">
  <rdfs:subClassOf rdf:resource="#Humain">
  <daml:disjointWith rdf:resource="#Homme">
</daml:Class>
```

On explique que la classe *Humain* a deux sous-classes disjointes : *Homme* et *Femme*.

```
<daml:Class rdf:ID="Père">
  <rdfs:sameClassAs>
    <daml:Restriction daml:minCardinality="1">
      <daml:onProperty rdf:resource="#possèdeEnfants"/>
      <daml:toClass rdf:resource="#Homme"/>
    </daml:Restriction>
  </rdfs:subClassOf>
</daml:Class>
```

On dit ici que la classe des *Pères* est précisément la classe des *Hommes* pour lesquels la cardinalité de l'attribut *possèdeEnfants* vaut au moins 1.

DAML-S

DAML-S signifie **DAML-Services** : il s'agit de développer un langage de description des capacités et des propriétés des services Web. Il serait ainsi possible d'automatiser la recherche, la découverte, l'utilisation et l'interconnexion des services Web. DAML-S est construit au-dessus de DAML+OIL ou de ses descendants.

Actuellement, DAML-S est toujours en développement, mais un document de travail a été publié en octobre 2002 [**DAML-S**].

Avec DAML-S, il s'agit de décrire *ce qu'*un service Web peut faire, et non pas *comment* il le fait [**ISWC2002**]. Voyons plus précisément où se situe ce langage...

Avec des langages comme DAML+OIL, il est possible de décrire de façon précise les contenus Web. Actuellement, on assiste au développement des services Web, qui permettent à des applications de s'échanger les contenus Web. Pour décrire le fonctionnement de tels services Web, il existe le langage **WSDL** (Web Services Description Language), qui permet de formaliser les couches de communication entre services Web sous forme XML. Avec DAML-S, on cherche à représenter *ce qui* est transmis, et *pourquoi* c'est transmis. Le *comment* est le domaine de WSDL et des langages de programmation.

DAML-S doit pouvoir assister les services Web dans les tâches suivantes :

- **découverte** de services Web, c'est à dire détermination des services qui répondent à un cahier des charges spécifié ;
- **invocation**, exécution d'un service Web donné ;
- **interopération** entre services Web, ce qui peut impliquer des processus de traduction qui préservent la sémantique ;
- **vérification** de propriétés de services ;
- **surveillance** de l'exécution d'une tâche complexe par un service ou un ensemble de services, de façon à détecter et expliquer les défaillances.

DAML-S est une ontologie DAML+OIL, dont la classe parente s'appelle **Service**. Une instance de cette classe est décrite par deux aspects :

- *"qu'est-ce que le service fournit aux agents ; qu'est-ce qu'il attend des agents ?"* Pour cela, un serice *présente* son **ServiceProfile** ;
- *"comment est-ce qu'il fonctionne ? quel est son modèle d'exécution ?"* À ces fins, un service est décrit par son **ServiceModel**.

Le **ServiceProfile** permet aux agents de découvrir et d'identifier un service. Il donne entre autres le nom du service, son niveau de qualité, le type de service rendu, etc.

Le **ServiceModel** permet aux agents de composer plusieurs services afin de résoudre un problème complexe, ou encore de surveiller le fonctionnement d'un service et d'établir des **diagnostics** en cas de défaillance. Le modèle d'exécution est décrit à l'aide de la classe **ProcessModel** qui fournit une ontologie des processus, qui décrit les services dans un modèle de machine à états. Il existe aussi une ontologie des ressources et du temps.

Cependant, les descriptions en termes de ServiceProfile et ServiceModel sont situées à un niveau abstrait, indépendant des protocoles de communication concrets qui permettent en pratique d'accéder aux services. C'est pourquoi on adjoint à cette description une *fondation*, le **ServiceGrounding**. Ce *grounding* fait la liaison entre le modèle et le profil d'une part, et les descriptions de protocoles en langage WSDL.

En résumé DAML-S et WSDL sont deux langages complémentaires pour la description de services Web : DAML-S en décrit la sémantique, et WSDL les protocoles de communication concrets et effectifs.

OWL

OWL signifie Web Ontology Language. Il est défini par le W3C dans [**W3-OWL**], mais une introduction peut être trouvée dans [**W3-OWL-GUIDE**]. C'est un langage très nouveau : à la date de rédaction de ce document, les spécifications définitives ne sont pas encore parues.

Comme les langages précédemment évoqués, OWL est construit au-dessus de RDF. Un document OWL est donc composé de triplets RDF, qui peuvent être écrits dans la syntaxe RDF de son choix. Au sein d'un document, les triplets RDF non définis dans la spécification OWL sont ignorés.

Tout comme DAML+OIL, mais contrairement à XML Schema, OWL permet de faire des raisonnements, des inférences sur l'ontologie. Ainsi, OWL permettra de conclure sur des faits implicites, alors que XML Schema ne sait traiter que l'explicite.

Il existe trois sortes d'OWL :

- **OWL Lite** : c'est une version d'OWL aux fonctionnalités réduites, mais qui restent quand même suffisantes pour bien des usages, comme la constitution de taxonomies ou de thesaurus ;
- **OWL DL** : il correspond exactement aux logiques de description. Ce langage est expressif, mais les procédures d'inférences sont complètes, et effectuables en un temps fini ;
- **OWL Full** : il donne à l'utilisateur une expressivité maximale, mais on n'a aucune garantie quant à la complétude et à la terminaison des procédures d'inférence.

Le Web étant un espace ouvert, on dit que les ontologies sont **monotones** : il n'est toujours possible que d'ajouter de l'information ; on ne peut jamais en *supprimer* des faits ou des conclusions.

Une ontologie OWL se compose des éléments suivants :

- en-têtes optionnels : commentaire, version, importation d'ontologie ;
- éléments de classe ;
- éléments de propriétés ;
- instances.

OWL sépare l'univers en deux :

- d'une part le domaine des **types de données** (types de données de schémas XML) ;
- d'autre part les **objets**, membres de classes OWL ou RDF.

Les classes sont soit anonymes, soit désignées par leur URI. Il existe deux classes particulières, `owl:Thing` et `owl:Nothing` : tout objet est instance d'`owl:Thing`, et `owl:Nothing` n'a aucune instance. Pour définir une classe, on peut :

- utiliser des opérateurs de relation et de subsomption entre classes : `subClassOf`, `disjointWith`, `disjointUnionOf`, `sameClassAs`, `equivalentTo` ;
- combiner des attributs ci-dessus grâce à des opérateurs booléens : `owl:intersectionOf`, `owl:unionOf`, `owl:complementOf` ;
- procéder à l'énumération exhaustive de ses instances ;
- définir la classe par restriction sur une propriété d'une classe préexistante (*"la classe des instances de C dont l'attribut A vaut..."*) grâce à : `owl:allValuesFrom`, `owl:hasValue`, `owl:someValuesFrom`, `owl:cardinality`, `owl:maxCardinality`, `owl:minCardinality`.

La définition des propriétés se fait :

- en tant que sous-classes de `ObjectProperty` ou `DatatypeProperty` grâce à l'attribut `rdfs:subPropertyOf` ;
- par restriction du domaine d'application (`rdfs:domain`, i.e. l'ensemble des classes à qui cette propriété peut s'appliquer) ou du domaine de variation (`rdfs:range`, i.e. l'ensemble des valeurs que peut prendre la propriété) ;
- par des relations entre propriétés : `owl:samePropertyAs`, `owl:equivalentTo`, `owl:inverseOf` ;
- en tant que sous-classe d'une des sous-classes particulières prédéfinies : `owl:TransitiveProperty`, `owl:SymmetricProperty`, `owl:FunctionalProperty`, `owl:InverseFunctionalProperty`. Ainsi, on peut facilement doter une propriété de caractéristiques bien particulières.

On vient de le voir, en OWL, la définition des classes se fait à peu près comme en DAML+OIL. Mais voyons ce qu'apporte ce langage pour la définition de propriétés, grâce à un exemple inspiré de [**W3-OWL-GUIDE**] :

```
<owl:ObjectProperty rdf:ID="situéDans">
  <rdf:type rdf:resource="owl:TransitiveProperty" />
  <rdfs:domain rdf:resource="owl:Thing" />
  <rdfs:range rdf:resource="#Lieu" />
</owl:ObjectProperty>

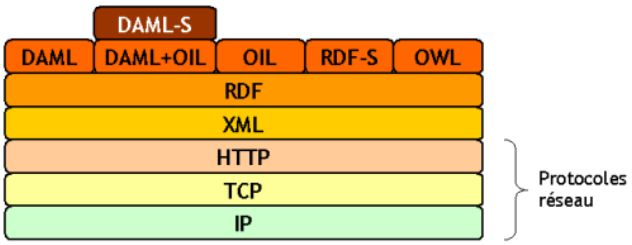
<Region rdf:ID="Orsay">
  <locatedIn rdf:resource="#IleDeFrance" />
</Region>

<Region rdf:ID="LIMSI">
  <locatedIn rdf:resource="#Orsay" />
</Region>
```

On indique tout d'abord que la propriété `situéDans` est transitive. Après les deux assertions suivantes (*"Orsay est situé en Île de France"* et *"le LIMSI est situé à Orsay"*), on peut conclure sur : *"le LIMSI est situé en Île de France"*.

Conclusion sur les travaux du W3C et consorts

Le schéma ci-contre montre la superposition des protocoles et langages pour l'échange d'ontologies entre agents.



Le W3C a défini plusieurs langages de description ontologiques dans le cadre de ses travaux sur le **Web sémantique**. Voici ce qu'il est possible d'en retenir de façon synthétique :

- RDF Schema est trop limité car il est impossible de faire des inférences ou du raisonnement ;
- DAML+OIL et OWL sont très complets, mais difficiles à implémenter ;
- OWL Lite est un langage suffisant dans bien des cas, et bien plus facile à implémenter que DAML+OIL.

Équipes

Les équipes qui travaillent sur les langages de communication entre agents sont généralement des groupements à but non lucratif, formés par des acteurs majeurs du secteur. On peut citer, entre autres :

- **W3C** : le World Wide Web consortium s'occupe du développement du Web. En ce sens, il favorise les recherches sur le Web sémantique : XML, RDF, RDF Schema, DAML+OIL, OWL ;
- **FIPA** : la Foundation for Intelligent Physical Agents a entre autres défini un langage de communication entre agents qui porte son nom (FIPA ACL) ;
- Groupe de travail de **DAML** : il s'agit d'une branche de la **DARPA** (Defense Advanced Research Project Agency, qui dépend du **DoD**, Department of Defense américain), qui s'occupe du développement du langage DAML ;
- **DAML-S Coalition** : ce groupement est constitué des personnes qui travaillent sur le projet DAML-S, voir [**ISWC2002**]. On peut trouver une **liste de leurs publications**, et ils sont présentés sur le **site de DAML**.

Cependant, d'autres équipes travaillent sur des thèmes connexes :

- **The Logic Group** de l'université de **Stanford** travaille sur les agents, et en particulier sur des thèmes proches de l'initiative KSE et du langage KIF ;
- **Yannis Labrou** des **Fujitsu Laboratories of America** a travaillé sur KQML et est lié à la FIPA ;
- **Tim Finnin** travaille entre autres dans le domaine des **agents** à l'**université du Maryland**, avec son collègue **Yun Peng** ;
- **Mamadou Tadiou Kone** travaille sur les agents et les langages de communication entre agents au **Japan Advanced Institute of Science and Technology** ;
- Le groupe **IAM** de l'**université de Southampton** travaille sur **DAML et les agents**, ainsi que sur les **grilles** à bases d'agents.

Liens : petite bibliographie commentée

Les liens suivants sont donnés par ordre d'apparition dans l'exposé :

[IEEE99]

The current landscape of Agent Communication Languages, Yannis Labrou, Tim Finin et Yun Peng. Mars 1999.

Cet article de 1999 dresse un panorama des langages de communication entre agents à cette date. Après avoir posé la problématique de la communication entre agents (*tous les agents doivent parler un langage commun pour pouvoir se comprendre*), il présente les deux ACLs principaux en 1999 : KQML puis l'ACL de la FIPA. Il évoque également le futur des systèmes multi-agents (code écrit en Java, possibilité de définir un ACL au-dessus des normes du **W3C**.

[SEARLE69]

Speech acts. An essay in the philosophy of language, John Searle. Cambridge University Press, London, 1969.

John Searle essaie de conceptualiser le langage humain en dégageant des actes de langage (en anglais, *speech acts*). Le livre de Searle n'est pas disponible en ligne, mais l'historique de cette théorie est

disponible à l'université John Hopkins : **The John Hopkins Guide to Literary Theory & Criticism : Speech Acts**. On peut également citer une **interview en français** avec John Searle, où il évoque la théorie des actes de langage.

[FIPA00061]

The FIPA ACL Message Structure Specifications, 6 décembre 2002.

Ce document décrit de façon normative la structure des messages de l'ACL de la FIPA : il s'agit essentiellement d'une description des divers attributs qui composent un message. Les performatifs ne sont pas décrits ici, mais dans [**FIPA00037**].

[FIPA00037]

FIPA Communicative Act Library Specification, 6 décembre 2002.

Il s'agit de la définition formelle des messages de l'ACL de la FIPA ainsi que de tous les performatifs. Chaque performatif est défini grâce au langage SL (Semantic Language), qui est lui-même décrit en annexe.

[FIPA00008]

FIPA SL Content Language Specification, 6 décembre 2002.

Le langage SL (Semantic Language) est à l'origine écrit sous forme mathématique pour décrire de façon formelle la sémantique de l'ACL de la FIPA. Mais ce document décrit une syntaxe basée sur des s-expressions (donc de type Lisp) pour le langage SL, qui peut alors être utilisé en tant que langage de contenu des messages écrits dans l'ACL de la FIPA.

[W3C-7]

Le W3C en 7 points.

Cette page liste les buts du W3C : accès universel, Web sémantique, confiance, interopérabilité, évolutivité, décentralisation, multimédia plus *cool*.

[W3-RDF-PRIMER]

RDF Primer, document de travail du W3C, 23 janvier 2003.

Ce document se veut une introduction au Resource Description Framework (RDF). Il part du but premier de RDF : la rédaction de métadonnées à propos de pages Web, et présente de nombreux exemples illustrés de schémas. Il donne des liens vers les documents plus formels de spécification de RDF. En outre, il décrit la syntaxe XML pour RDF (tout en soulignant que ce n'est pas la seule syntaxe envisageable). Enfin, il présente RDF Schema, et fait une revue des principales applications actuelles de RDF : RSS, Dublin Core, etc.

[W3-RDF-CONCEPTS]

Resource Description Framework (RDF) : Concepts and Abstract Syntax, document de travail du W3C, 23 janvier 2003.

On a droit ici à une présentation précise des concepts du RDF : définition des termes utilisés pour parler de RDF, signification des éléments du langage, et syntaxe abstraite (indépendante de toute syntaxe concrète, XML ou autre).

[W3-RDF-XML]

RDF/XML Syntax Specification (Revised), document de travail du W3C, 23 janvier 2003.

Ce document normalise la méthode de représentation des descriptions RDF à l'aide de documents XML. Entre autres, il définit l'espace de noms à utiliser pour RDF (`http://www.w3.org/1999/02/22-rdf-syntax-ns`), souvent désigné par le préfixe `rdf`.

[W3-RDFS]

RDF Vocabulary Description Language 1.0 : RDF Schema, document de travail du W3C, 23 janvier 2003.

Ce document décrit le langage RDF Schema, qui permet de décrire des vocabulaires (ou schémas) RDF. RDF Schema ne permet pas de décrire des ressources concrètes comme des pages Web, mais se situe à

un niveau d'abstraction supplémentaire. Il permet de décrire les vocabulaires qui à leur tour seront utilisés pour décrire ces ressources concrètes.

[DAML-ONT]

DAML-ONT Initial Release. Octobre 2000.

Ce document présente le langage DAML initial, avant son intégration avec OIL. DAML est un langage de description d'ontologies portant sur des classes de ressources et de propriétés. Cette spécification présente un vocabulaire RDF qui permet de définir les ontologies (relations entre classes et propriétés, restrictions sur les propriétés, etc.).

[W3-DAML+OIL]

DAML+OIL Reference Description. Mars 2001.

Ce document spécifie le langage DAML+OIL, langage ontologique du W3C, construit au-dessus de RDF, et à la sémantique clairement définie. Il s'agit d'un guide parfois un peu informel de DAML+OIL, dont le but est de permettre une approche relativement facile du langage.

[DAML-S]

DAML-S 0.7 Draft Release. Octobre 2002.

Ce document donne l'état de développement du langage de description de services Web DAML-S. Il présente des documents introductifs ainsi que de nombreux exemples.

[ISWC2002]

DAML-S : Web Service Description for the Semantic Web, The First International Semantic Web Conference (ISWC), Sardaigne, juin 2002.

Il s'agit d'une présentation générale du langage DAML-S dans son contexte : quels sont les objectifs du langage (description de services Web indépendamment de leur implémentation), quels sont les relations entre DAML-S et le langage WSDL.

[W3-OWL]

Web Ontology Language (OWL) Reference Version 1.0, document de travail du W3C, 12 novembre 2002.

Il s'agit de la spécification du langage OWL, non encore définitive. OWL est un langage d'ontologie pour le Web, basé sur RDF, et inspiré de DAML+OIL. Ce document est censé servir de guide de référence pour le langage OWL. Pour une introduction à OWL, il vaut mieux d'abord consulter [**W3-OWL-GUIDE**].

[W3-OWL-GUIDE]

Web Ontology Language (OWL) Guide Version 1.0, document de travail du W3C, 10 février 2003.

C'est une très bonne introduction, non formelle, à la fois aux problèmes du Web sémantique, des ontologies, et du langage OWL en particulier. Elle décrit les méthodes de définition de classes de propriétés et d'attributs, les types de données et l'instanciation des classes. De nombreux exemples concrets sont donnés tout au long de la discussion.

[W3-OWL-SYN]

Web Ontology Language (OWL) Abstract Syntax and Semantics, document de travail du W3C, 3 février 2003.

Ce document définit de façon très formelle la syntaxe du langage OWL, en partant de la comparaison avec les langages DAML+OIL et RDF Schema. C'est un document très théorique, même mathématique.

[W3-OWL-FEAT]

Web Ontology Language (OWL) : Overview, document de travail du W3C, 10 février 2003.

Il s'agit d'une présentation générale du langage OWL et de ses trois sous langages. Il donne des définitions informelles des constructions disponibles dans chacun des langages, en faisant également le lien avec RDF Schema.